Relationship between Array and Pointer in C language

22 April 2026 20:17

In C, arrays and pointers are tightly coupled. Understanding 1D and 2D arrays through pointers means understanding how memory is laid out and how pointer arithmetic maps to indexing.

1D Array using pointers

A 1d array is a contiguous block of memory.

```
int A[] = {10, 20, 30, 40, 50};
```

In the memory:

A[0]	A[1]	A[2]	A[3]	A[4]
10	20	30	40	50

- A stores the base address.
- A[i] equivalent *(A + i)

This is the key pointer relation:

A[i] = *(A + i)

```c
#include<stdio.h>
int main(int argc, char const *argv[])
{
    int A[] = {10, 20, 30, 40, 50};
    int *p = A; //points to the first element
    int i;
    for(i = 0; i < 5; i++){
        printf("Value of A[%d] : %d, Address of A[%d]: %u\n", i, *(p+i), i, (p + i));
    }
    return 0;
}
```

de-reference operator: it means value at the address.

pointer variable

no asterisk means, we get address.

address of a cell = (base_address + i * size_of_datatype)

Output:

```
Value of A[0] : 10, Address of A[0]: 3084908848
Value of A[1] : 20, Address of A[1]: 3084908852
Value of A[2] : 30, Address of A[2]: 3084908856
Value of A[3] : 40, Address of A[3]: 3084908860
Value of A[4] : 50, Address of A[4]: 3084908864
```

int type in C language has size 4-bytes

Array store its elements in contiguous memory locations.

address of a cell = (base_address + i * size_of_datatype)

3084908848 + 3 * 4 = 3084908848 + 12 =

3084908860

```c
#include<stdio.h>
```

```c
int main(int argc, char const *argv[])
{
    int A[] = {10, 20, 30, 40, 50};
    int *p = A; //points to the first element
    int i;
    for(i = 0; i < 5; i++){
        printf("Value of A[%d] : %d, Address of A[%d]: %u\n", i, *(p), i, p);
        p++;
    }
    return 0;
}
```

de-reference operator

pointer variable

when we are incrementing pointer, it is pointing to next memory allocation.

content of address hold by the pointer.

content of pointer

dynamic

```
Output:
   Value of A[0] : 10, Address of A[0]: 111148320
   Value of A[1] : 20, Address of A[1]: 111148324
   Value of A[2] : 30, Address of A[2]: 111148328
   Value of A[3] : 40, Address of A[3]: 111148332
   Value of A[4] : 50, Address of A[4]: 111148336
```

```c
#include<stdio.h>
int main(int argc, char const *argv[])
{
    int A[] = {10, 20, 30, 40, 50};
    int *p = A; //points to the first element
    int i;
    for(i = 0; i < 5; i++){
        printf("Value of A[%d] : %d, Address of A[%d]: %u\n", i, *(p), i, p);
        p++;
    }
    p--; //get back to the last location
    printf("\nDecrementing the pointer:\n");
    for(i = 0; i < 5; i++){
        printf("Value of A[%d] : %d, Address of A[%d]: %u\n", i, *(p), i, p);
        p--;
    }
    return 0;
}
```

```
Output:
   Value of A[0] : 10, Address of A[0]: 1728052816
   Value of A[1] : 20, Address of A[1]: 1728052820
   Value of A[2] : 30, Address of A[2]: 1728052824
   Value of A[3] : 40, Address of A[3]: 1728052828
   Value of A[4] : 50, Address of A[4]: 1728052832

   Decrementing the pointer:
   Value of A[0] : 50, Address of A[0]: 1728052832
   Value of A[1] : 40, Address of A[1]: 1728052828
   Value of A[2] : 30, Address of A[2]: 1728052824
```

```
Value of A[3] : 20, Address of A[3]: 1728052820
Value of A[4] : 10, Address of A[4]: 1728052816
```

2D Array Using Pointers A 2D array is stored in a row major order. (Row by row.)

```
int A[2][3] = {
    {1, 2, 3},
    {4, 5, 6}
};
```

Memory layout:

A[0][0]	A[0][1]	A[0][2]	A[1][0]	A[1][1]	A[1][2]
1	2	3	4	5	6

A[i][j] can be written like *(*(A+i)+ j)

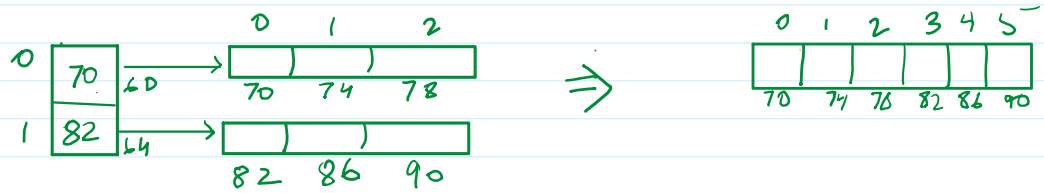
pointer to the 1st row

(A+i) → ith row

*(A+i) → base address of ith row.

*(A+i)+j → address of ith row & jth column or the cell.

((A+i)+j) → accessing the value of ith row & jth column.




```
# include<stdio.h>
int main(int argc, char const *argv[])
{
    int A[2][3] = {
        {1, 2, 3},
        {4, 5, 6}
    };
    int i, j;
    for(i = 0; i < 2; i++){
        for(j = 0; j < 3; j++){
            printf("A[%d][%d]: %d, Address A[%d][%d]: %u\n", i, j, *(*(A+i)+j), i,
j, (*(A+i)+j));
        }
    }
    return 0;
}
```

1	2	3	4	5	6
36	40	44	48	52	56

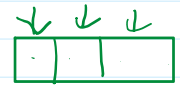
base address

data

```
Output:
   A[0][0]: 1, Address A[0][0]: 4167040336
   A[0][1]: 2, Address A[0][1]: 4167040340
   A[0][2]: 3, Address A[0][2]: 4167040344
   A[1][0]: 4, Address A[1][0]: 4167040348
   A[1][1]: 5, Address A[1][1]: 4167040352
   A[1][2]: 6, Address A[1][2]: 4167040356
```

the data stored in 2D array in contiguous format.

```
A[0][2]: 3, Address A[0][2]: 4167040344
A[1][0]: 4, Address A[1][0]: 4167040348
A[1][1]: 5, Address A[1][1]: 4167040352
A[1][2]: 6, Address A[1][2]: 4167040356
```



no. of columns (C)

baseaddress + i * sizeof(data_type) * + j * sizeof(data type)

baseaddrey + sizeof(data_type)(i.C + j)

```
*(*(A+i)+j)
```

$36 + 4(0 \times 3 + 0) = 36$

$36 + 4(0 \times 3 + 1) = 40$

$36 + 4(0 \times 3 + 2) = 44$

$36 + 4(1 \times 3 + 0) = 48$

$36 + 4(1 \times 3 + 1) = 52$

$36 + 4(1 \times 3 + 2) = 56$

address A[i][j] = base_address + sizeof(data_type)(i × C + j)

where, i = row index
j = column index
C = No. of columns in Array.

```c
# include<stdio.h>
int main(int argc, char const *argv[])
{
    int A[2][3] = {
        {1, 2, 3},
        {4, 5, 6}
    };
    int i, j;
    int (*p)[3]; // pointer to array of 3 integers
    p = A;
    for(i = 0; i < 2; i++){
        for(j = 0; j < 3; j++){
            printf("A[%d][%d]: %d, Address A[%d][%d]: %u\n", i, j,
*(*(p+i)+j), i, j, (*(p+i)+j));
        }
    }
    return 0;
}
```

Output:

```
A[0][0]: 1, Address A[0][0]: 2569009344
A[0][1]: 2, Address A[0][1]: 2569009348
```

```
A[0][2]: 3, Address A[0][2]: 2569009352
A[1][0]: 4, Address A[1][0]: 2569009356
A[1][1]: 5, Address A[1][1]: 2569009360
A[1][2]: 6, Address A[1][2]: 2569009364
```

Another example using Flat pointer:

```c
# include<stdio.h>
int main(int argc, char const *argv[])
{
    int A[2][3] = {
        {1, 2, 3},
        {4, 5, 6}
    };
    int i, j;
    int *p = &A[0][0]; // Flat pointer(linear access)
    for(i = 0; i < 6; i++){
            printf("%d, Address: %u\n", *(p+i), (p+i));
    }
    return 0;
}
```

$\boxed{\text{address } A[i] = \text{base_address} + \text{sizeof(data_type)}(i)}$

internally the above formula is applied.

this is used to access linearly

Output:

```
1, Address: 4001365456
2, Address: 4001365460
3, Address: 4001365464
4, Address: 4001365468
5, Address: 4001365472
6, Address: 4001365476
```

value stored on the address